When Should the Second Drone Start

Ganipisetty Mahaswi

August 2025

Introduction

In current two-drone missions, the second drone waits until it reaches full capacity (e.g., 4 people) and then delivers payloads by finding the shortest path. Waiting for the full batch introduces delays and negatively impacts overall mission time.

To minimize waiting time, we propose determining when the second drone should launch earlier, based on an equation that accounts for both movement and delivery times.

Proposed Algorithm

The second drone should start when either:

1. The batch reaches maximum capacity.
2. The remaining mission time is less than or equal to the estimated delivery time of the current batch.

Formally, let:

- C = max capacity
- k = number of people found until that moment of time
- t_i^{move} = time to move to point i
- t_i^{deliver} = time to deliver payload at point i
- T_{remain} = Total Mission Time - Elapsed Time

The launch condition is:

$$T_{\text{remain}} \leq \sum_{i=1}^{k} (t_i^{\text{move}} + t_i^{\text{deliver}}) \quad \text{or} \quad k \geq C$$

Reason for this formula: Before Drone 2 finishes delivering the current batch, Drone 1 may complete its path and detect additional points. This formula ensures Drone 2 launches

without unnecessary idle time, considering both movement and delivery separately.And the second drone continues if more human are detected by the time time it delivers the payloads to the number of people found else it returns ! And clearly it reduces the time but the problem is with the ability of the drone to receive the signals during the motion and some more testcases have to be considered!

Proof of Superiority

Let $T_{\text{wait-full}}$ be the time if the drone waits for full capacity:

$$T_{\text{wait-full}} = t_{\text{wait}} + \sum_{i=1}^{\min(k,C)} (t_i^{\text{move}} + t_i^{\text{deliver}}) + \text{time for next batches}$$

Let T_{equation} be the time using our proposed formula:

$$T_{\text{equation}} = \sum_{i=1}^{\min(k,C)} (t_i^{\text{move}} + t_i^{\text{deliver}}) + \text{time for next batches}$$

Since $t_{\text{wait}} \geq 0$, we have:

$$T_{\text{equation}} \leq T_{\text{wait-full}}$$

Launching based on this equation avoids idle waiting and guarantees faster or equal delivery time compared to waiting for full capacity.

Code Implementation

```
import time
from itertools import permutations

MAX_CAPACITY = 5
TOTAL_MISSION_TIME = 600  # seconds
elapsed_time = 0

point_queue = []

def estimated_delivery_time(batch):
    return sum(point['move_time'] + point['delivery_time'] for
       point in batch)

def shortest_path(batch):
    if len(batch) <= 1:
        return batch
    best_order = batch
    min_time = estimated_delivery_time(batch)
```

```
    for perm in permutations(batch):
        t = estimated_delivery_time(perm)
        if t < min_time:
            min_time = t
            best_order = perm
    return best_order

while elapsed_time < TOTAL_MISSION_TIME:
    if point_queue:
        batch_size = min(len(point_queue), MAX_CAPACITY)
        batch = point_queue[:batch_size]
        rem_time = TOTAL_MISSION_TIME - elapsed_time
        est_time = estimated_delivery_time(batch)

        if rem_time <= est_time or batch_size >= MAX_CAPACITY:
            optimized_batch = shortest_path(batch)
            print(f"Launching␣Drone␣2␣with␣batch:␣{[p['id']␣for
               ␣p␣in␣optimized_batch]}")
            delivery_time = estimated_delivery_time(
               optimized_batch)
            time.sleep(delivery_time)
            elapsed_time += delivery_time
            point_queue = point_queue[batch_size:]
        else:
            time.sleep(1)
            elapsed_time += 1
```

Conclusion

Our goal is to minimize overall mission time by launching the second drone based on the proposed equation. By considering both movement and delivery times separately, and starting the drone when the formula is satisfied, we ensure faster and more efficient payload delivery in time-critical missions.